

Timer Setup for Capture

TIM2 32-bit input capture on PB10, register by register

SWEN 563 / CMPE 663 / EEEE 663 — Real-Time & Embedded Systems · Project 1 course handout. This handout ships with the assignment, because the timers lecture (L7) is on Sep 14. The handout is self-contained. The lecture repeats this material.

References: RM0434 (TIM2 chapter), DS11929 (pin AF tables), Project 1 spec §7.1 (SENSE input = Arduino **D5** = **PB10** = **TIM2_CH3**, **AF1** — authoritative).

1. Why input capture (and not "read CNT in a loop")

A program can poll `GPIOB->IDR`, find an edge, and read a counter. But tens of microseconds can pass before the software finds the edge. The error also changes with the other work of the CPU. This is *software-induced error*, and a 1 μ s instrument cannot carry it.

Input capture moves the timestamp into hardware. The timer monitors the pin. At the instant of the edge, the timer copies the free-running counter CNT into a capture register (CCR3).

Software can collect the value microseconds *or milliseconds* later, because the timestamp stays frozen in CCR3. Loop latency does not corrupt the measurement. It only limits how quickly you can collect results (demo D8 tests that limit).

TIM2 is important because it is the **only 32-bit** general-purpose timer on this part. At 1 MHz, the wrap period of a 16-bit counter is 65.5 ms. The wrap period of a 32-bit counter is ~71.6 minutes, which is far past any acquisition in the 500 Hz to 10 kHz band.

Know these numbers (Q&A candidate).

The prescaler sets the count rate:

```
counter clock = f_CK_PSC / (PSC + 1)
```

The trap, in writing (spec §5.6): at 64 MHz, a 1 MHz tick needs $PSC = 63$, **not 64**. $PSC = 64$ gives 984.6 ns ticks. Each interval then reads ~1.5% short, ~985 counts for a true 1000 μ s.

Demo step **D4** exists to catch exactly this error. The operator moves the generator to 1.010 kHz, and the bin cluster shows at ~990. Derive the value. Do not inherit it.

2. The register walk

There are six configuration steps, in the sequence below. The code uses read-modify-write for all registers, but not for the registers that this project owns fully. TIM2 is yours alone, and you share the GPIO ports with the provided drivers (§7.3).

Step 0 — Clocks: RCC

A peripheral does not operate until its clock is on. TIM2 is on the APB1 bus. GPIOB is on the AHB2 bus.

```
RCC->APB1ENR1 |= RCC_APB1ENR1_TIM2EN;
RCC->AHB2ENR  |= RCC_AHB2ENR_GPIOBEN;
(void)RCC->APB1ENR1;                /* read back: guarantee the enable has
                                     landed before the first TIM2 access */
```

With $PPRE1 = /1$ (the provided `clock_init()` keeps this value), the TIM2 kernel clock is **64 MHz**.

Step 1 — Pin: PB10 to alternate function 1, with pull-down

Set three GPIOB registers, and touch **only the pin-10 bits**. GPIOB is a shared port, and the console owns PB6/PB7. A wholesale write here stops your demo before the end:

```
/* MODER: 10 = alternate function */
GPIOB->MODER = (GPIOB->MODER & ~GPIO_MODER_MODE10) | GPIO_MODER_MODE10_1;

/* AFRH (AFR[1]): pins 8..15; AF1 = TIM2_CH3 on PB10 */
GPIOB->AFR[1] = (GPIOB->AFR[1] & ~GPIO_AFRH_AFSEL10)
    | (1u << GPIO_AFRH_AFSEL10_Pos);

/* PUPDR: 10 = pull-down — MANDATORY (spec section 7.1). The generator's
 * Output-OFF relay leaves the line FLOATING; a mid-rail Schmitt input can
 * chatter and defeat POST (R1). The pull-down also holds a loose lead
 * quietly low. */
GPIOB->PUPDR = (GPIOB->PUPDR & ~GPIO_PUPDR_PUPD10) | GPIO_PUPDR_PUPD10_1;
```

Step 2 — Prescaler: PSC, then force it to load with EGR

PSC is a **preloaded** register. A value that you write takes effect only at the next *update event*. If you write PSC and then start a capture, your first acquisition counts at the unchanged rate. Cause an *update event* immediately:

```
TIM2->PSC = 63u;          /* 64 MHz / (63+1) = 1 MHz -> 1 us tick */
TIM2->EGR = TIM_EGR_UG;    /* update generation: latch PSC now */
```

(ARR resets to 0xFFFFFFFF on TIM2 — the full 32-bit range. Do not change it.)

Step 3 — Channel 3 as input: CCMR2

Channels 1/2 are in CCMR1. **Channels 3/4 are in CCMR2** — a frequent incorrect-register bug. Two fields for CH3:

- CC3S = 01 — channel 3 becomes an **input**, and the timer maps it to TI3 (the PB10 pin). The reset value 00 means "output", and then a capture cannot occur.
- IC3F — input filter: the edge must be stable for N samples before it counts. 0011 (8 samples at 64 MHz ≈ 125 ns) rejects glitches and adds no measurable latency. The filter delays *all* edges equally, and thus the intervals (differences) do not change.

```
TIM2->CCMR2 = (TIM2->CCMR2 & ~(TIM_CCMR2_CC3S | TIM_CCMR2_IC3F))
    | (1u << TIM_CCMR2_CC3S_Pos)    /* input, TI3 */
    | (3u << TIM_CCMR2_IC3F_Pos);   /* filter: 8 samples */
```

Step 4 — Edge select and enable: CCER

CC3P/CC3NP = 00 selects **rising** edges. This is the reset default, but set it explicitly, because an explicit value is auditable. CC3E connects the capture.

```
TIM2->CCER = (TIM2->CCER & ~(TIM_CCER_CC3P | TIM_CCER_CC3NP))
    | TIM_CCER_CC3E;
```

Step 5 — Run: CR1

```
TIM2->CR1 |= TIM_CR1_CEN;          /* counter runs from here on */
```

3. Capturing

Poll the status register for the capture flag of channel 3. Then read the timestamp. **A read of CCR3 clears CC3IF by hardware** — a manual flag write is not necessary on this path:

```
uint32_t wait_edge(void)
{
    while ((TIM2->SR & TIM_SR_CC3IF) == 0u) {
```

```

        /* the R1 POST window check belongs here when POST calls this
           loop. A deadline in a run is optional: the spec does not
           grade a mid-acquisition signal loss (R12 scope note). */
    }
    return TIM2->CCR3;          /* read clears CC3IF */
}

```

An interval is the difference of two consecutive captures:

```

uint32_t t0 = wait_edge();      /* both edges captured THIS run (R5) */
uint32_t t1 = wait_edge();
uint32_t interval_us = t1 - t0; /* unsigned subtraction: correct even
                                when CNT wrapped between edges */

```

Because of this unsigned-subtraction property, the code needs no special condition for a wrapped counter. $(\text{uint32_t})(t1 - t0)$ is exact modulo 2^{32} . It fails only when the true interval is more than the full 71.6-minute span. That condition does not occur in this instrument.

Overcapture: SR.CC3OF

If a second edge occurs before you read CCR3, the hardware sets the **overcapture** flag CC3OF, and CCR3 holds the *newest* edge. The capture path loses one edge with no indication.

R2 permits no missed edges from 500 Hz to 10 kHz. At 10 kHz, edges come at 100 μs intervals. Thus each slow operation in your capture loop shows here (an OLED write takes $\sim 200 \mu\text{s}$).

Check the flag. A set flag shows a design error, not a condition to hide:

```

if ((TIM2->SR & TIM_SR_CC3OF) != 0u) {
    TIM2->SR = ~TIM_SR_CC3OF; /* rc_w0: write 0 to clear, 1s elsewhere */
    /* an edge was lost: your loop is too slow – fix the loop, don't
       massage the data (R5: every sample disposed, honestly) */
}

```

Two rules have this cause. R9 mandates the static ACQUIRING screen, and R10 bans UART traffic during an acquisition. D8 (10 kHz, $N = 1000$, TOTAL must equal exactly 1000) is the test that shows this fault.

Interrupt alternative

The spec permits a poll of CC3IF for this project (spec §5.3). The interrupt variant sets $\text{TIM2->DIER} \mid= \text{TIM_DIER_CC3IE}$, enables TIM2_IRQn in the NVIC, and reads CCR3 in $\text{TIM2_IRQHandler}()$. The registers and the math are the same. The ISR must stay short: read the value, keep it, and set a flag for the main loop.

The 663 duty-cycle extension (spec §8) uses the interrupt variant with **both-edge** capture: set CC3P and CC3NP together in CCER, and classify each edge in the ISR from the pin level in GPIOB->IDR .

4. Checklist and first-light test

Initialization sequence (a missing step is the most frequent cause of a "no captures ever" report):

- 1 RCC->APB1ENR1 TIM2EN, RCC->AHB2ENR GPIOBEN (+ read-back)
- 2 PB10: MODER=AF, AFRH=AF1, PUPDR=pull-down
- 3 PSC = 63, then EGR = UG
- 4 CCMR2: CC3S=01, IC3F set
- 5 CCER: CC3E (rising: CC3P=CC3NP=0)
- 6 CR1: CEN
- 7 Loop: poll SR CC3IF → read CCR3 → subtract → dispose (bin/OUT tally)

First light, at the bench: use the generator preset from §7.2 (1.000 kHz square, 3.3 Vpp, +1.65 V offset, **High-Z**). Connect the oscilloscope, through a tee, to CH1 as ground truth. Print ten consecutive intervals *after* the capture, and do not print during the capture (R10).

All values should read $1000 \pm 1 \mu\text{s}$. If the values are near ~985, read §1 again. If the values are near ~2000 or irregular, check CC30F. A set flag shows that your loop loses edges. If there are no values at all, do the checklist from top to bottom.

First light, at home: a generator is not necessary. Bit-bang a square wave on a spare GPIO (Arduino D2 = PD12) from a SysTick-paced loop, and jumper it to D5 (the LOOPBACK self-stimulus, spec §6). LOOPBACK validates this capture path, with no bench equipment.

5. What this handout deliberately does not give you

The graded work is the logic that disposes of each sample (histogram bins, OUT_LOW/OUT_HIGH, the first-interval rule), the non-blocking R1 POST window, the deferred-I/O structure, and the statistics engine. The registers above give you "edges timestamped at 1 μs ". The instrument is yours.